

# DIGITAL SIGNAL PROCESSING LAB REPORT

Âu Lê Quang Trí - Student ID

Nguyễn Lê Khải Tú - Student ID

October 5, 2026

## Overview

This report summarizes the results of the Digital Signal Processing laboratory exercises, specifically focusing on generating, manipulating, and analyzing discrete-time signals and systems using Scilab. The report covers Lab 1 and Lab 2.

## 1 Introduction

The laboratory exercises provide practical experience with continuous-time to discrete-time signal conversion, signal properties, and elementary discrete-time operations. By utilizing Scilab, theoretical concepts such as sampling, quantization, time-shifting, folding, and signal arithmetic are modeled and visualized to deepen our mathematical understanding.

## 2 Objectives

The objectives achieved upon completing these laboratory sessions include:

- Utilizing Scilab for vector and matrix initializations.
- Understanding the sampling theorem and the quantization process (ADC procedure).
- Visualizing discrete signals via functions such as `plot2d3` and utilizing graph annotations.
- Implementing fundamental operations on discrete-time signals, including addition, multiplication, time-shifting, and time-reversal.

- Extracting the even and odd components of a given discrete-time sequence.

## 3 Methodology

The exercises were conducted on a Windows environment using the Scilab numerical computing software. Mathematical formulas from the time-domain representation of discrete signals were translated into Scilab scripts. Vectorized operations and padding techniques were applied where necessary to align discrete sequences prior to manipulation.

## 4 Practical Contents

### 4.1 Lab 1

#### Exercise 1.1

**Question:** Using the operators in vector and matrix to do the following tasks:

- Create a vector in the form of  $(x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1)$  where  $x_1, x_2, x_3, x_4$  are components of vector  $x = 1 : 4$ .
- Create a vector in the form of  $(x_1y_1, x_2y_2, x_3y_3, x_4y_4)$  where  $x_1 \dots x_4$  and  $y_1 \dots y_4$  are components of vector  $x = 1 : 4$  and vector  $y = 5 : 8$ , respectively.
- Create a vector in the form of  $(\sin(x_1), \sin(x_2), \dots, \sin(x_{10}))$  where  $x$  is a vector of 10 values linearly chosen in the interval  $[0, \pi]$ .

**Explanation:** This exercise introduces element-wise vector operations in Scilab.

- For the first task, adding a scalar to a vector adds that scalar to every element identically:  $x + 1 \Rightarrow [2, 3, 4, 5]$ .
- The second task requires element-by-element multiplication (the Hadamard product). In Scilab, this is executed using the `.*` operator, multiplying elements at corresponding indices:  $x .* y \Rightarrow [1 \times 5, 2 \times 6, 3 \times 7, 4 \times 8]$ .
- The third task utilizes the `linspace(0, %pi, 10)` function to generate an array of  $N = 10$  linearly spaced points. The `sin()` function is then mapped across the generated vector space.

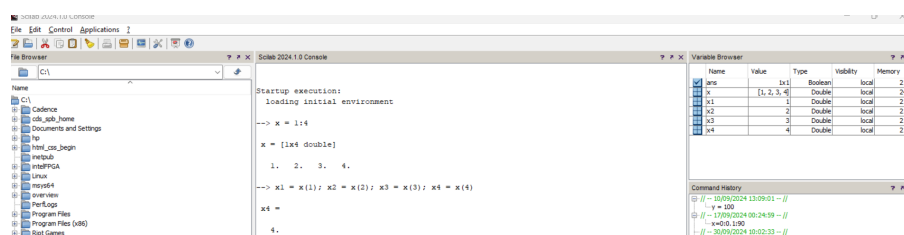


Figure 1: Result of Ex 1.1 - Part 1

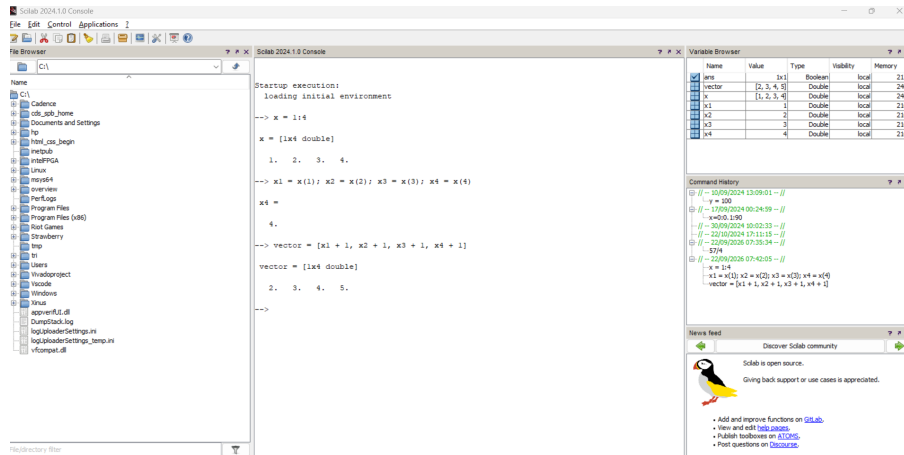


Figure 2: Result of Ex 1.1 - Part 2

## Exercise 1.2

**Question:** Consider the analog signal  $x_a(t) = 3 \sin(100\pi t)$ .

- Use Scilab to draw  $x_a(t)$  in 5 periods.
- Determine the discrete-time signal  $x(n)$  of the signal  $x_a(t)$  that is sampled with a sampling rate  $F_s = 300$  samples/s.
- Determine the discrete-time signal  $x(n)$  and determine its periodic property. If  $x(n)$  is a periodic signal, determine the frequency and period of  $x(n)$ . Then, use Scilab to draw  $x(n)$  in 5 periods.
- Determine the quantized signal  $x_q(n)$  if  $\Delta = 0.1$  using the truncated method. Then, draw  $x_q(n)$  in 5 periods. All figures must be displayed in a single window.

**Explanation:** Given  $x_a(t) = 3 \sin(100\pi t)$ , the analog frequency is  $F_0 = 50$  Hz. By applying the sampling theorem with  $F_s = 300$  Hz, the sampling interval is  $T_s = 1/F_s$ . Substituting  $t = nT_s$ :

$$x(n) = x_a(nT_s) = 3 \sin\left(100\pi \frac{n}{300}\right) = 3 \sin\left(\frac{\pi}{3}n\right) \quad (1)$$

A discrete sinusoid is periodic if its digital frequency  $f = \frac{F_0}{F_s} = \frac{50}{300} = \frac{1}{6}$  is a rational number. Because  $f = \frac{1}{6}$ ,  $x(n)$  is periodic. The fundamental period is determined by  $N = \frac{K}{f}$  where  $K$  is the smallest integer yielding an integer  $N$ . Thus, for  $K = 1$ , the discrete period is  $N = 6$  samples.

For quantization, we apply a truncation method with step size  $\Delta = 0.1$ . The truncated value  $x_q(n)$  is obtained via the floor operation after normalizing by  $\Delta$ :

$$x_q(n) = \Delta \left\lfloor \frac{x(n)}{\Delta} \right\rfloor = 0.1 \lfloor 10 \cdot x(n) \rfloor \quad (2)$$

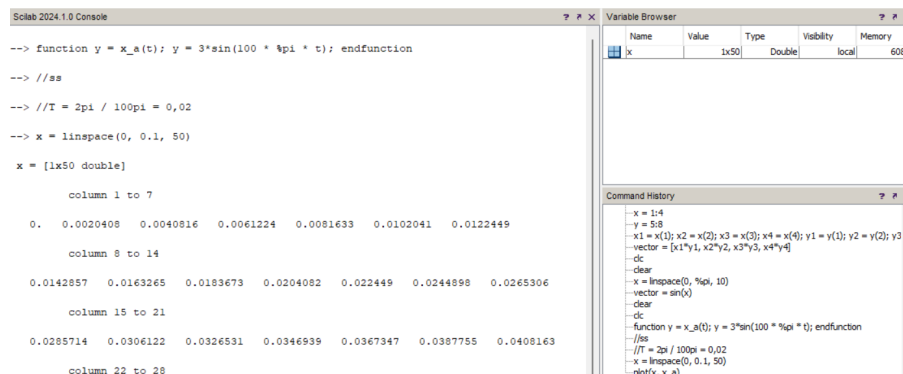


Figure 3: Result of Ex 1.2 - Analog and Discrete Signal

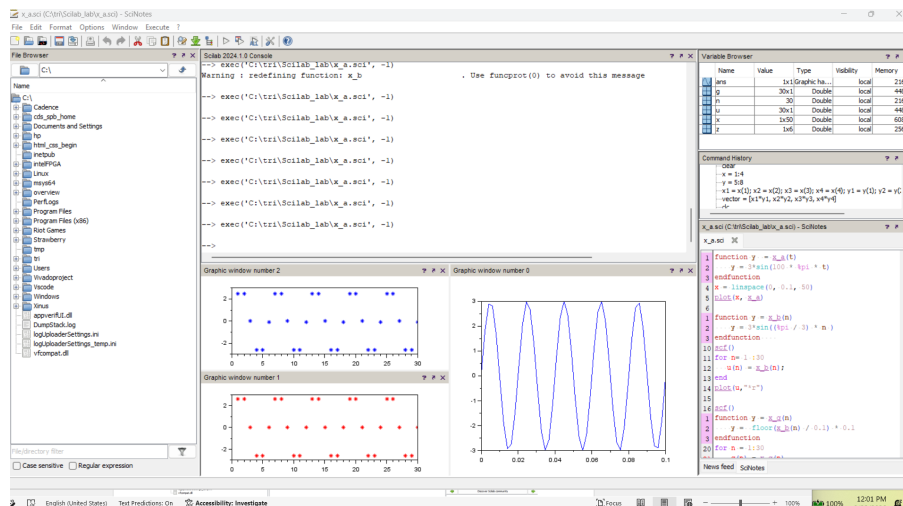


Figure 4: Result of Ex 1.2 - Quantized Signal and Errors

## 4.2 Lab 2

### Exercise 1: Investigate Basic Functions

**Question:** Let's investigate the following functions on Scilab and briefly report their functionalities and how to use it: `plot2d3`, `min`, `max`, `subplot`, `title`, `xlabel`, `ylabel`, `bool2s`, `deff`.

**Explanation:** This exercise evaluates fundamental plotting and computational functions:

- `plot2d3(x, y)`: Plots vertical lines (stems) representing discrete-time samples. It is strictly used for discrete signals to highlight values at integer indices.
- `max()` and `min()`: Calculates the maximum/minimum elements in a matrix or vector array, highly useful for determining normalization constants or quantization step limits.
- `subplot(m, n, p)`: Divides the graphic window into an  $m \times n$  matrix of sub-windows, selecting the  $p^{th}$  window for the active plot.

- `title()`, `xlabel()`, `ylabel()`: These functions add textual annotations to the graph. `title()` sets the main heading, while `xlabel()` and `ylabel()` label the respective axes, which is crucial for identifying discrete components properly.
- `bool2s()`: Converts a boolean array containing %T and %F into a numerical array of 1s and 0s. This is mathematically equivalent to the Heaviside step function evaluating a logic condition.
- `deff()`: Defines inline custom functions on the fly, similar to lambda expressions. Syntax: `deff("[y] = f(x)", "y = ...")`.

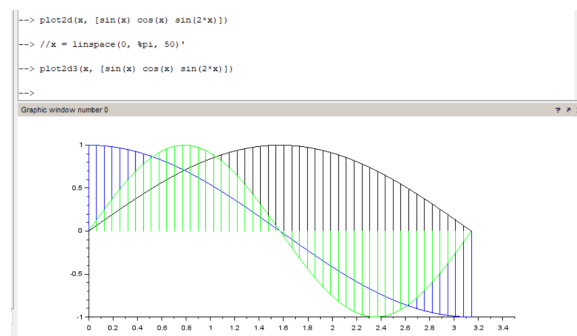


Figure 5: plot2d3 Usage

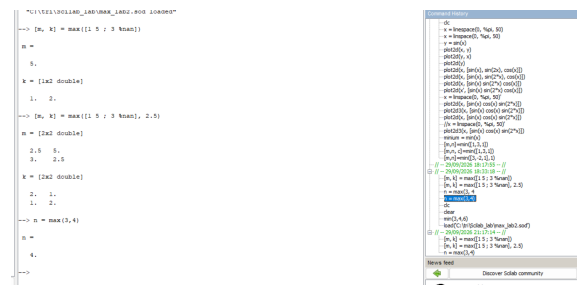


Figure 6: max() Usage

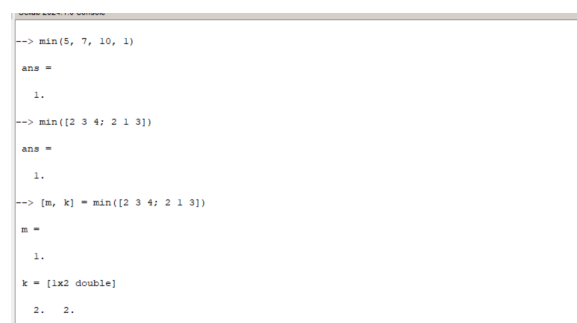


Figure 7: min() Usage

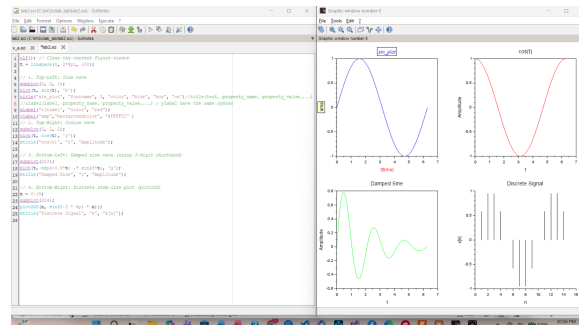


Figure 8: subplot() + title() + xlabel() + ylabel() Usage

```
--> deff("y = f(x)", " y = 3*x + 1");

--> f(5)

ans =

    16.
```

Figure 9: deff Usage

## Exercise 2 & Exercise 3

**Question:** Try the following scripts on Scilab and report your understanding after observing the output.

- `n = -5:5; msignal = bool2s(n >= 0); plot2d3(n, msignal)`
- `n = -5:5; msignal = bool2s(n == 0); plot2d3(n, msignal)`

### Explanation:

- **Exercise 2:** The logical condition `n >= 0` evaluates to True for  $n \geq 0$  and False otherwise. `bool2s` transforms these booleans into numeric 1s and 0s. The output precisely replicates the mathematical definition of a **unit step sequence**  $u(n)$ :

$$u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases} \quad (3)$$

- **Exercise 3:** The equality `n == 0` is true only at the origin point  $n = 0$ . Consequently, applying `bool2s` creates a sequence that represents the **unit sample sequence (discrete Dirac delta function)**  $\delta(n)$ :

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \quad (4)$$

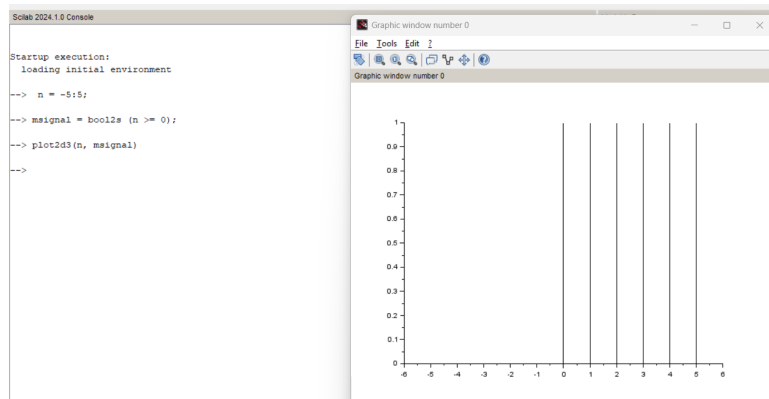


Figure 10: Result of Ex 2 - Unit Step Sequence  $u(n)$

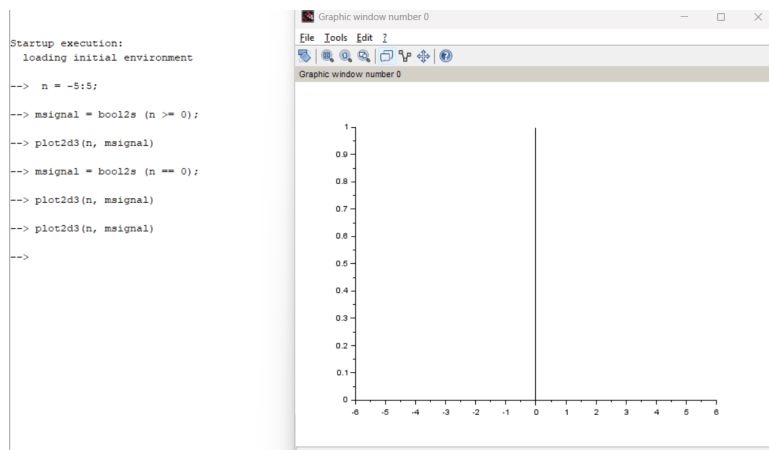


Figure 11: Result of Ex 3 - Unit Sample Sequence  $\delta(n)$

## Exercise 4: Unit Ramp Signal

**Question:** Use Scilab to draw the unit ramp signal  $u_r(n)$  for  $n = -5 : 5$ .

**Explanation:** The discrete unit ramp signal is mathematically defined as the integral of the unit step function, or more precisely:

$$u_r(n) = n \cdot u(n) = \begin{cases} n, & n \geq 0 \\ 0, & n < 0 \end{cases} \quad (5)$$

In Scilab, this is implemented natively by executing an element-wise product between the index space array  $n$  and the previously defined unit step array.

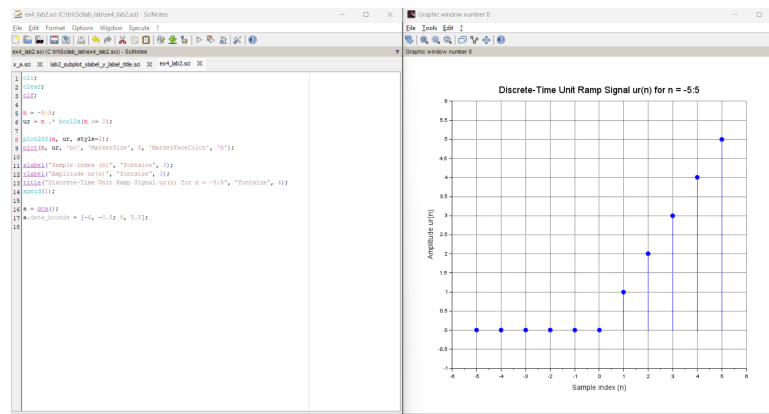


Figure 12: Result of Ex 4 - Unit Ramp Signal

### Exercise 5: Even and Odd Components

**Question:** Given a discrete-time signal  $x(n] = \{1, 3 \uparrow, -2\}$ . Use Scilab to draw the signal  $x(n)$ , the odd signal component  $x_o(n)$ , and the even signal component  $x_e(n)$ .

**Explanation:** The arrow signifies the origin index  $n = 0$ . The sequence is thus explicitly indexed as  $x(-1) = 1, x(0) = 3, x(1) = -2$ . According to signal properties, any arbitrary sequence can be perfectly decomposed into an even-symmetric component  $x_e(n)$  and an odd-symmetric component  $x_o(n)$ . We first evaluate the folded (time-reversed) sequence  $x(-n)$ . Using the decomposition formula:

$$x_e(n) = \frac{x(n) + x(-n)}{2}, \quad x_o(n) = \frac{x(n) - x(-n)}{2} \quad (6)$$

Calculations yield  $x_e(-1) = -0.5, x_e(0) = 3, x_e(1) = -0.5$ , and correspondingly  $x_o(-1) = 1.5, x_o(0) = 0, x_o(1) = -1.5$ . Scilab strictly mirrors these mathematical operations linearly. Therefore, the resulting sequences are:

$$x_e(n) = \{-0.5, 3 \uparrow, -0.5\}, \quad x_o(n) = \{1.5, 0 \uparrow, -1.5\} \quad (7)$$

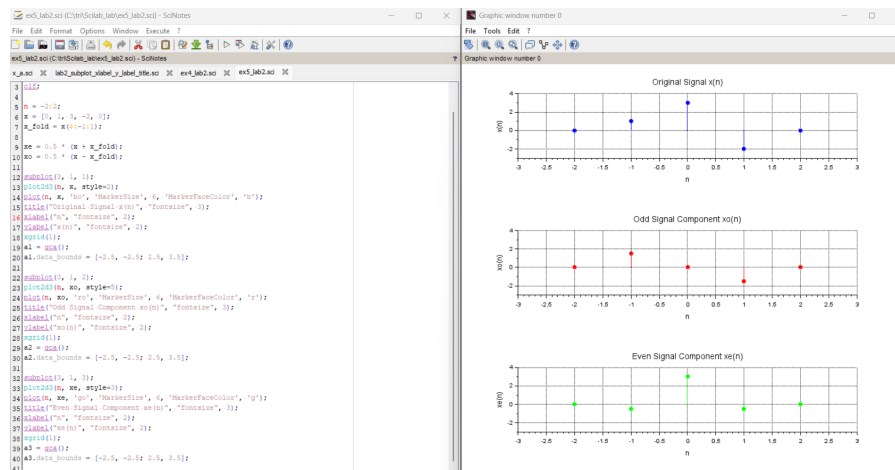


Figure 13: Result of Ex 5 - Even and Odd Decomposition

## Exercise 6 & 7: Addition and Multiplication of Signals

### Question:

- **Exercise 6:** Given two discrete-time signals  $x_1(n) = \{0 \uparrow, 1, 3, -2\}$  and  $x_2(n) = \{0, 1 \uparrow, 2, 3\}$ . Determine  $y(n) = x_1(n) + x_2(n)$ .
- **Exercise 7:** Given the same signals, determine  $y(n) = x_1(n) \cdot x_2(n)$ .

**Explanation:** Because the two sequences span different bounded domain intervals ( $x_1(n)$  spans  $[0, 3]$  and  $x_2(n)$  spans  $[-1, 2]$ ), they cannot be directly operated on using primitive arrays. To implement a rigorous sample-by-sample operation, we determine the union of both index domains:  $[\min(n_{1_{\min}}, n_{2_{\min}}), \max(n_{1_{\max}}, n_{2_{\max}})] = [-1, 3]$ . Both vectors are then symmetrically zero-padded to conform to this unified domain grid. Once the sequences structurally align relative to the time origin  $n = 0$ :

$$y_{add}(n) = x_1(n) + x_2(n) = \{0, 1 \uparrow, 3, 6, -2\} \quad (8)$$

$$y_{mult}(n) = x_1(n) \cdot x_2(n) = \{0, 0 \uparrow, 2, 9, 0\} \text{ or simply } \{0 \uparrow, 2, 9\} \quad (9)$$

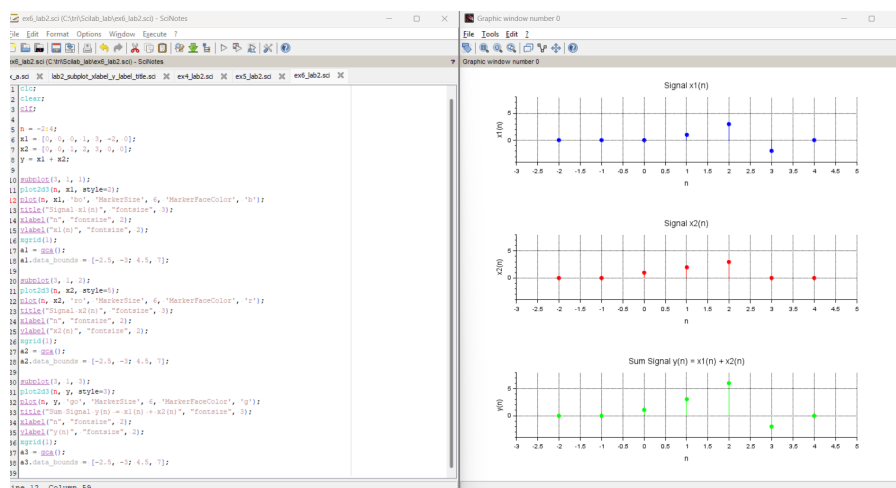


Figure 14: Result of Ex 6 - Addition of Signals

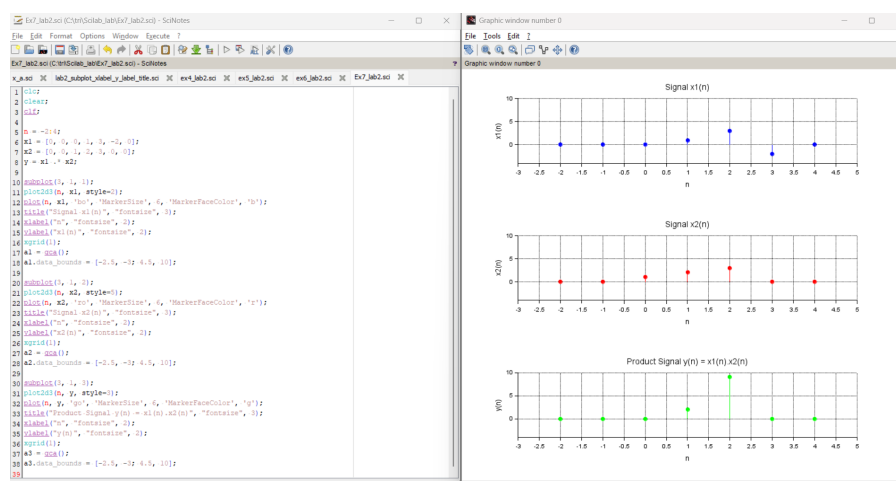


Figure 15: Result of Ex 7 - Multiplication of Signals

## Exercise 8: Signal Manipulations

**Question:** Given a discrete-time signal  $x(n] = \{1, -2, 3 \uparrow, 6\}$ . Determine the following signals and draw both original and manipulated signals: 1.  $y_1(n) = x(-n)$  2.  $y_2(n) = x(n + 3)$  3.  $y_3(n) = 2x(-n - 2)$

**Explanation:** For  $x(n)$ , the origin translates to  $x(-2) = 1, x(-1) = -2, x(0) = 3, x(1) = 6$ . We systematically apply independent operations on the sequence index variables:

1. **Folding (Time-Reversal):**  $y_1(n) = x(-n)$ . Every index evaluates to its symmetrical negative counterpart. Resulting indices:  $[-1, 2]$ .

$$y_1(n) = \{6, 3 \uparrow, -2, 1\} \quad (10)$$

2. **Advance (Time-Shift):**  $y_2(n) = x(n + 3)$ . The addition inside the argument denotes an advancement. The whole sequence is shifted to the left along the time axis by 3 sample units.

$$y_2(n) = \{1, -2, 3, 6, 0, 0 \uparrow\} \quad (11)$$

3. **Composite Operations:**  $y_3(n) = 2x(-(n + 2))$ . To evaluate this correctly: first perform folding to get  $x(-n)$ , then shift left by 2 to get  $x(-(n + 2))$ , and finally apply an amplitude scaling factor of 2.

$$y_3(n) = \{12, 6, -4, 2 \uparrow\} \quad (12)$$

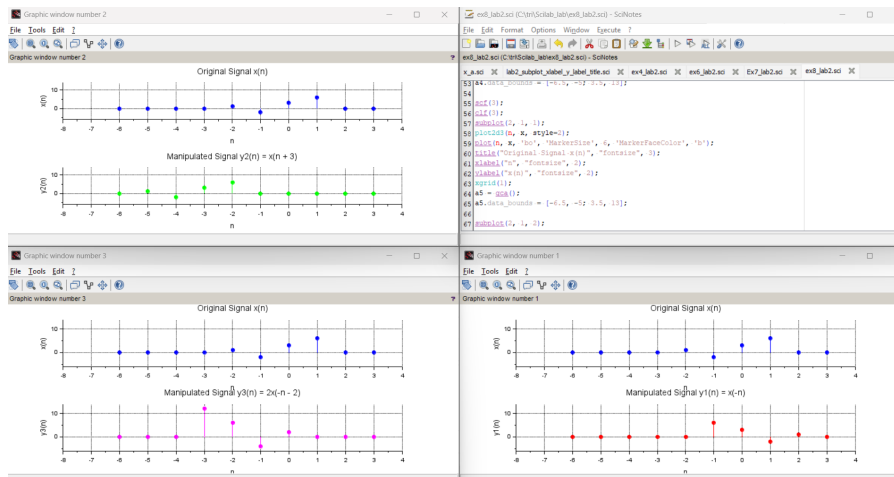


Figure 16: Result of Ex 8 - Signal Manipulations

**Additional exercise 1.1**

**Additional exercise 1.3**

**Additional exercise 1.6**

**Additional exercise 1.7**

**Additional exercise 1.9**

**Additional exercise 1.10**

## **5 Discussion**

The laboratory thoroughly consolidated the theoretical understanding of discrete-time signals using systematic computational approaches. Scilab proved to be a versatile platform, allowing direct vector mapping of discrete mathematical formulas and offering visual validations through its `plot2d3` subroutines. Specifically, manual implementations of zero-padding demonstrated exactly how sequences must be mutually aligned relative to an origin reference prior to algebraic operations.

## **6 References**

- Digital Signal Processing Lecture Notes - Ho Chi Minh City University of Technology.
- Scilab Online Manuals.